SVKM'S NMIMS
Deemed-to-be UNIVERSITY

Beretta

Design Document

An Explainable Automated Cyber-Insurance Underwriting and Continuous Monitoring Platform

Krish Mundhra (K041)
Harsh Rathod (K080)
Yugandhara Goud (K023)

Faculty Mentor: Prof. Rohit Suryawanshi

Mukesh Patel School of Technology Management & Engineering
SVKM's NMIMS, Mumbai

Academic year 2026-27

Contents

1. Introduction 4

1.1 Purpose 4

1.2 Scope 4

1.3 Notation 4

1.4 Design rules 4

2. System context 5

3. System architecture 6

3.1 Components 6

3.2 Deployment 7

3.3 Data flow 8

4. Users and use cases 9

4.1 The insured company 9

4.2 The insurer 10

5. Data design 11

6. Behaviour 12

6.1 Warranty state machine 12

6.2 Evidence status of a control 13

6.3 Quote and bind 14

6.4 Agent enrolment and a signed report 15

7. Algorithms 16

7.1 Premium calculation 16

7.2 Agent service loop 18

7.3 Fleet roll-up 18

7.4 Evidence resolution 19

8. References 19

1. Introduction

1.1 Purpose

This document records the design of Beretta, an explainable cyber-insurance platform for Indian small and medium enterprises (SMEs). It shows how the system is structured, how data moves through it, how its main objects change state, and how its main algorithms work. It supports the live demonstration and the project report, which gives the full explanation and results.

1.2 Scope

Beretta has two linked parts. In the presentation these are described as two engines:

- The risk engine gathers evidence about security controls and watches them after a policy is bound. It covers Engine A of the platform, together with the endpoint agent and the collector.
- The pricing engine calculates the premium once, at inception. It is Engine B of the platform (model V2).

The diagrams use the names in the code: Engine A, Engine B, Part 1 (the platform) and Part 2 (the agent and collector). The agent is shown on Windows, the set-up used in the demonstration.

1.3 Notation

Structural and behavioural diagrams follow UML 2.5.1 [1]. Data flow is shown with data-flow diagrams, and the document follows the viewpoint approach of IEEE 1016 [2]. Solid blue lines mark network calls between subsystems; dashed lines mark dependencies inside a subsystem.

1.4 Design rules

Five rules shape every design decision below:

- Monitoring never re-prices a bound policy. Agent and fleet data reach a bound policy only through the warranty state machine.
- The platform is the only warranty authority for linked companies. The collector displays its states.
- The agent is read-only. It never collects file contents, browsing, keystrokes, e-mail, passwords or clear-text user names.
- Every pricing number is kept in one parameter file with its unit, source, locator and a tag: published, derived or assumed.
- Simulated data is always labelled SIMULATED.

2. System context

Fig. 1 treats Beretta as a single process and shows the people and machines around it. The insured company is read-only and sees only its own data. The underwriter enters intake and prices, the administrator changes the model configuration, and the collector operator runs the monitoring server. Endpoints send only signed, read-only check results.

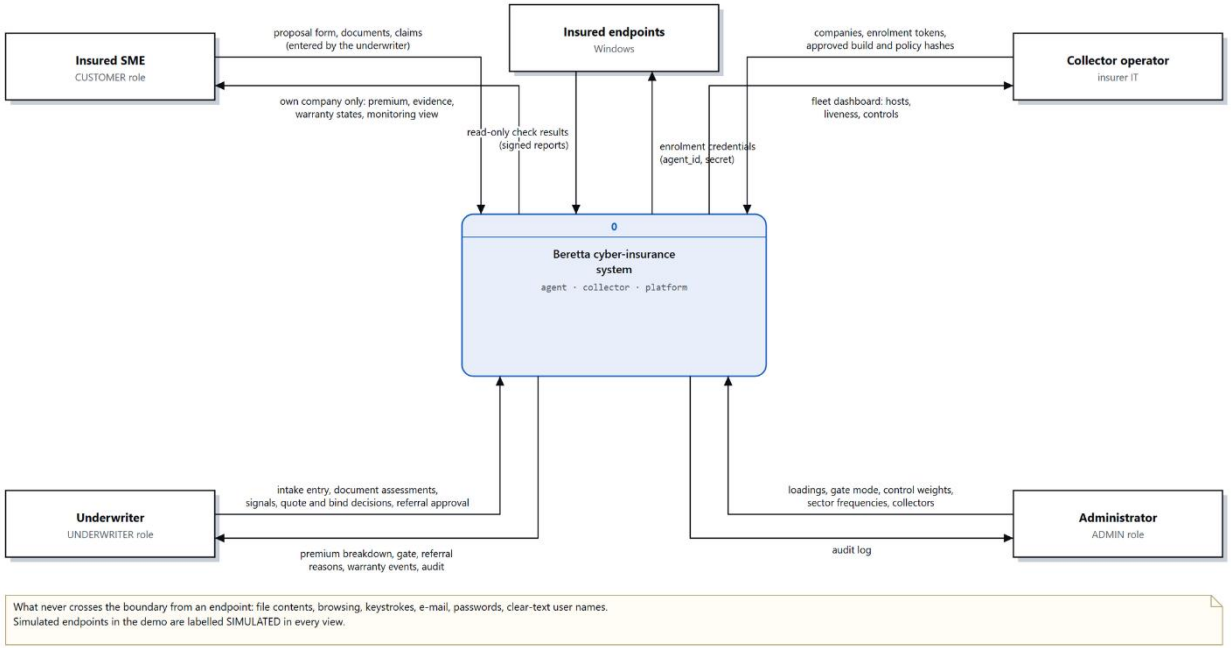


Fig. 1. Context data-flow diagram (level 0).

3. System architecture

3.1 Components

Fig. 2 shows the three subsystems. The agent sends signed reports to the collector over HTTPS with a pinned certificate authority. The collector evaluates them per company and sends one signed roll-up per linked company to the platform, which replies with the warranty states it holds.

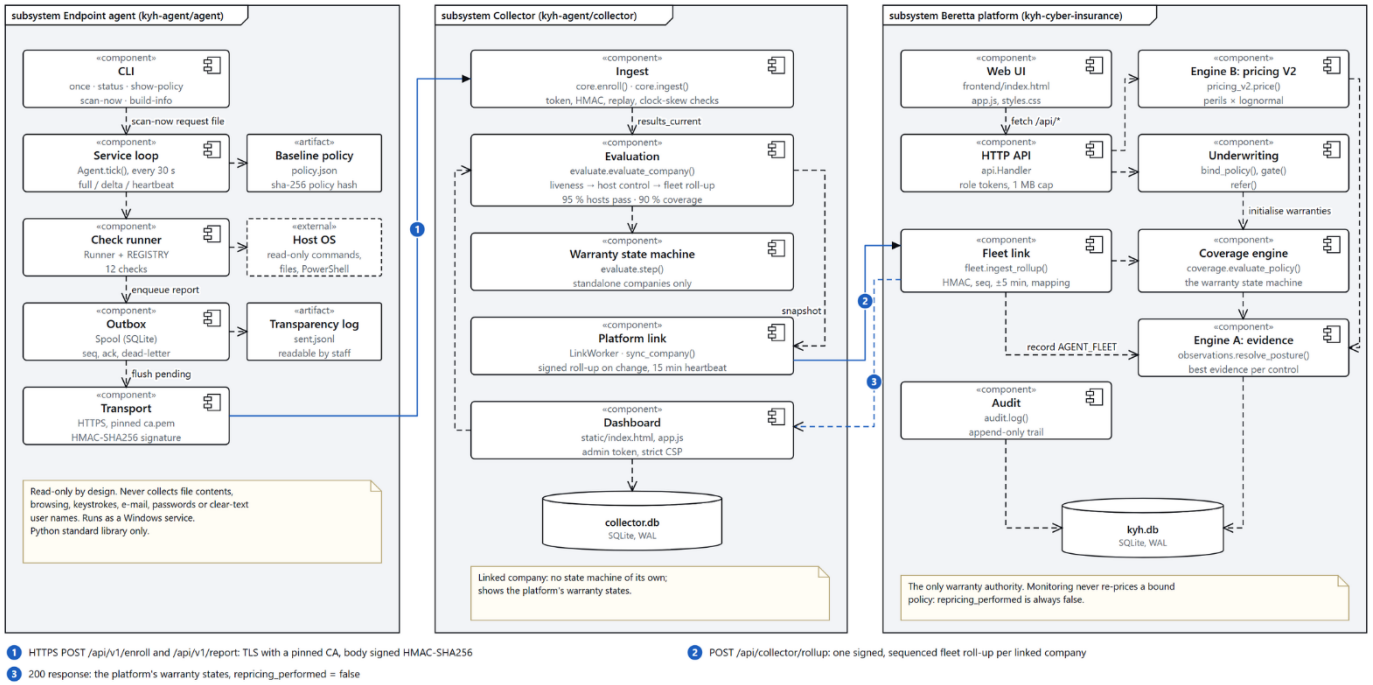


Fig. 2. UML component diagram of the three subsystems and how they connect.

3.2 Deployment

Fig. 3 shows where each part runs. In the demonstration all three nodes run on one laptop with four SIMULATED endpoints. A real Windows virtual machine can run the agent and report to the collector over a host-only network.

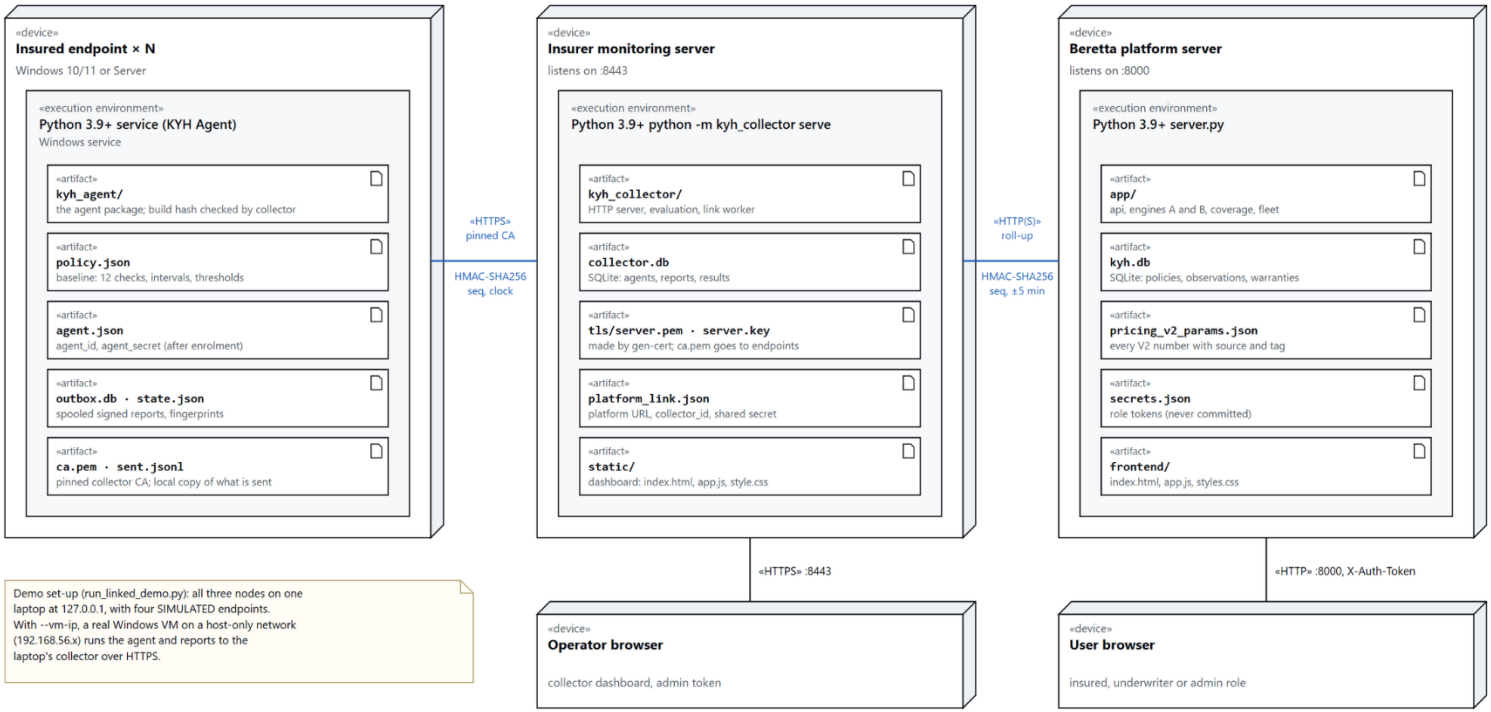


Fig. 3. UML deployment diagram.

3.3 Data flow

Fig. 4 follows evidence from an endpoint to a warranty state. Part 2 runs checks, validates and stores reports, evaluates the fleet and pushes a roll-up. Part 1 records it as evidence alongside the proposal form, prices the risk once, and then runs warranties on the bound policy.

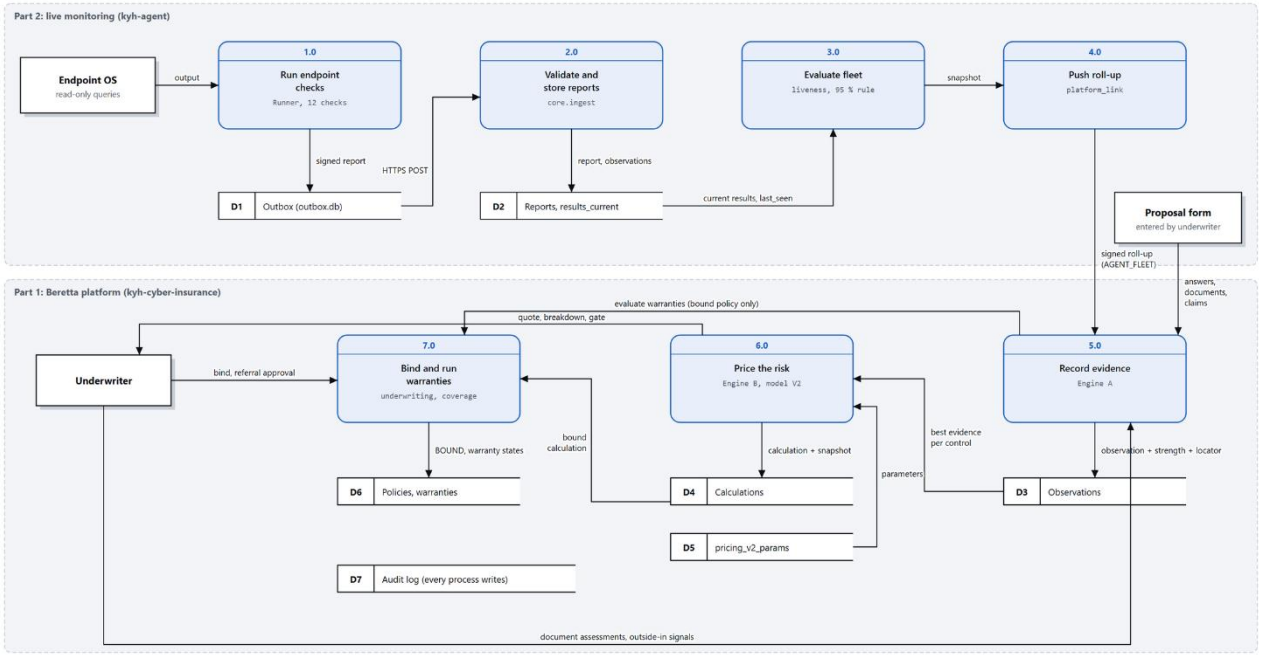


Fig. 4. Level-1 data-flow diagram: processes and data stores.

4. Users and use cases

4.1 The insured company

The customer role is read-only and limited to its own company. The IT administrator installs and enrols the agent, and can see exactly what it checks and sends.

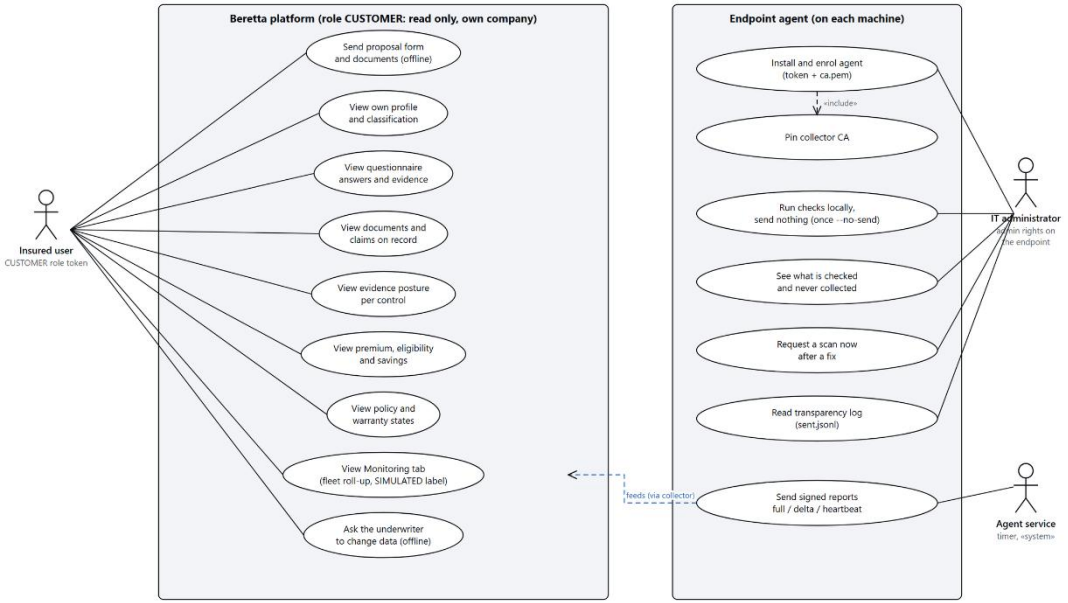


Fig. 5. Use-case diagram for the insured company.

4.2 The insurer

The underwriter enters intake, prices and binds. The administrator inherits those use cases and also changes the model configuration. The collector operator runs monitoring. Every change is audited.

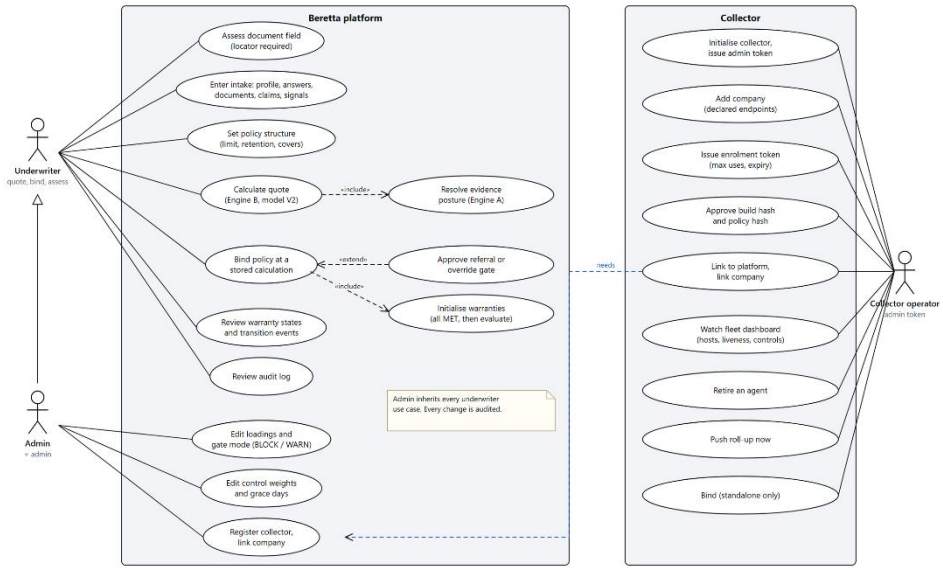


Fig. 6. Use-case diagram for the insurer.

5. Data design

A company has one policy. A policy has its coverages, its calculations (each with the full input snapshot and the model version) and its warranties, and each warranty has a history of state changes. Every observation records its control, source, locator, evidence strength and time. Every decision is written to the audit table.

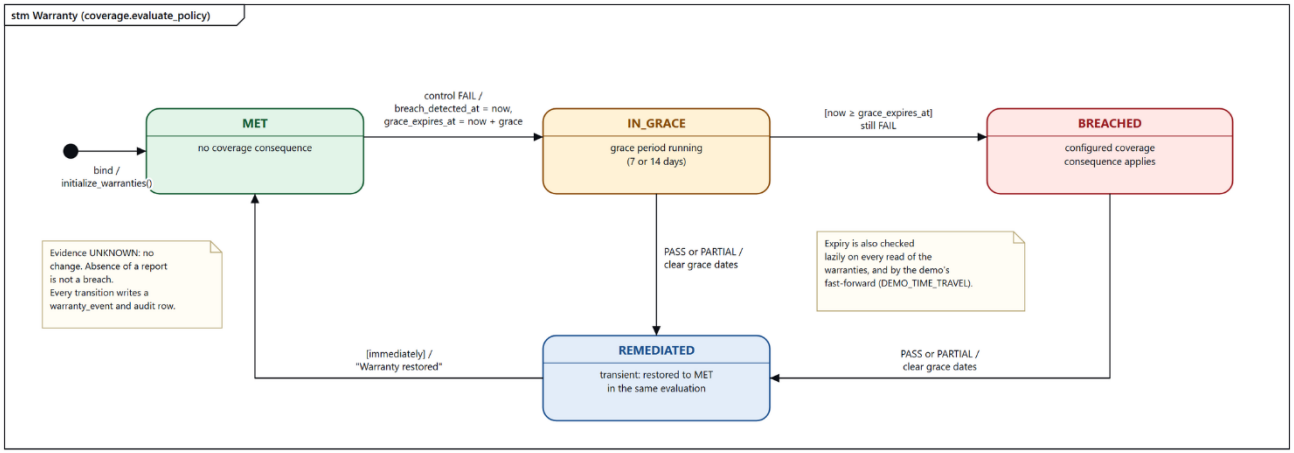


Fig. 7. Entity-relationship diagram of the platform database.

6. Behaviour

6.1 Warranty state machine

After bind, each warranty control is MET, IN_GRACE, BREACHED or REMEDIATED. A failing result starts a grace period of 7 or 14 days. Missing evidence is never a breach, and no transition changes the premium.



Warranty controls and their configured consequences (catalogue.py, illustrative wording)

Control	Grace	Coverage consequence while BREACHED
CTRL-MFA	7 days	Ransomware and cyber-extortion coverage head suspended
CTRL-BACKUP-TESTED	7 days	Data-restoration claims subject to 50 % co-insurance
CTRL-EDR	7 days	Incident-response deductible doubled
CTRL-PATCH-CRIT	14 days	Claims arising from unpatched critical CVEs subject to co-insurance
CTRL-AGENT-COVERAGE	14 days	Unmonitored endpoints treated as non-compliant with every warranty

What triggers an evaluation: any new observation for the company (questionnaire, document, fleet roll-up), bind itself, and the expiry check. The premium never changes.

Fig. 8. UML state machine of a platform warranty, with the controls and their illustrative consequences.

6.2 Evidence status of a control

The pricing model reads the best evidence for each control as VERIFIED, STATED, NOT_WORKING or UNKNOWN. Better evidence moves a control towards VERIFIED and never raises the premium.

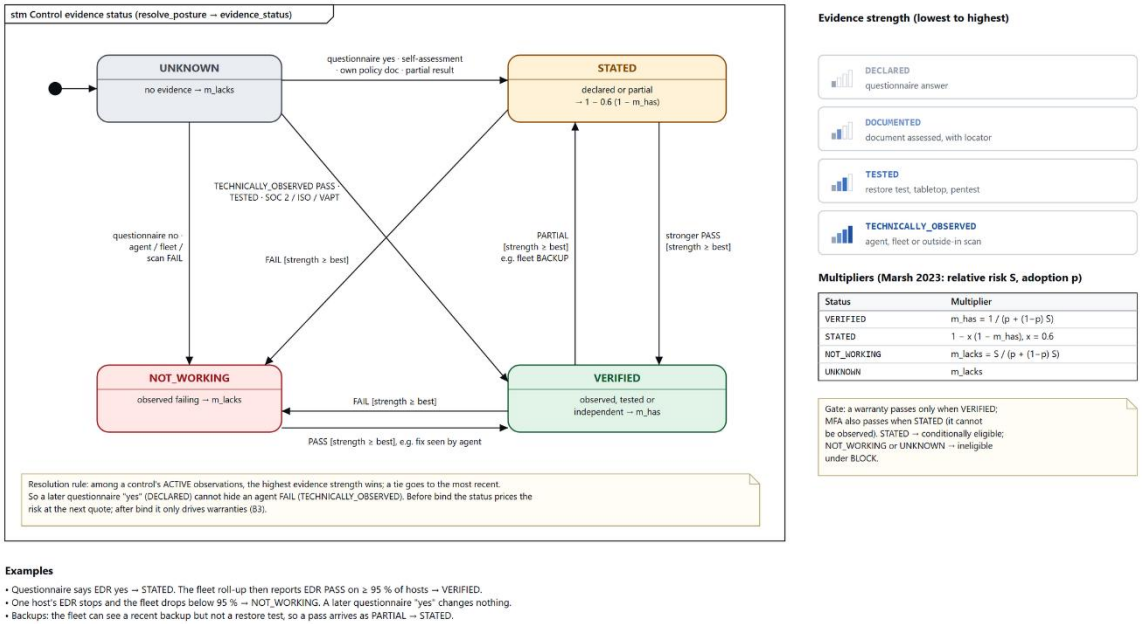


Fig. 9. State diagram of a control's evidence status.

6.3 Quote and bind

Fig. 10 shows a quote and a bind across the modules. A quote stores its calculation with a snapshot. Binding refuses a policy that is already bound or an ineligible quote under gate mode BLOCK, and a referred quote needs approval. At bind the warranties are created, all MET, and evaluated once.

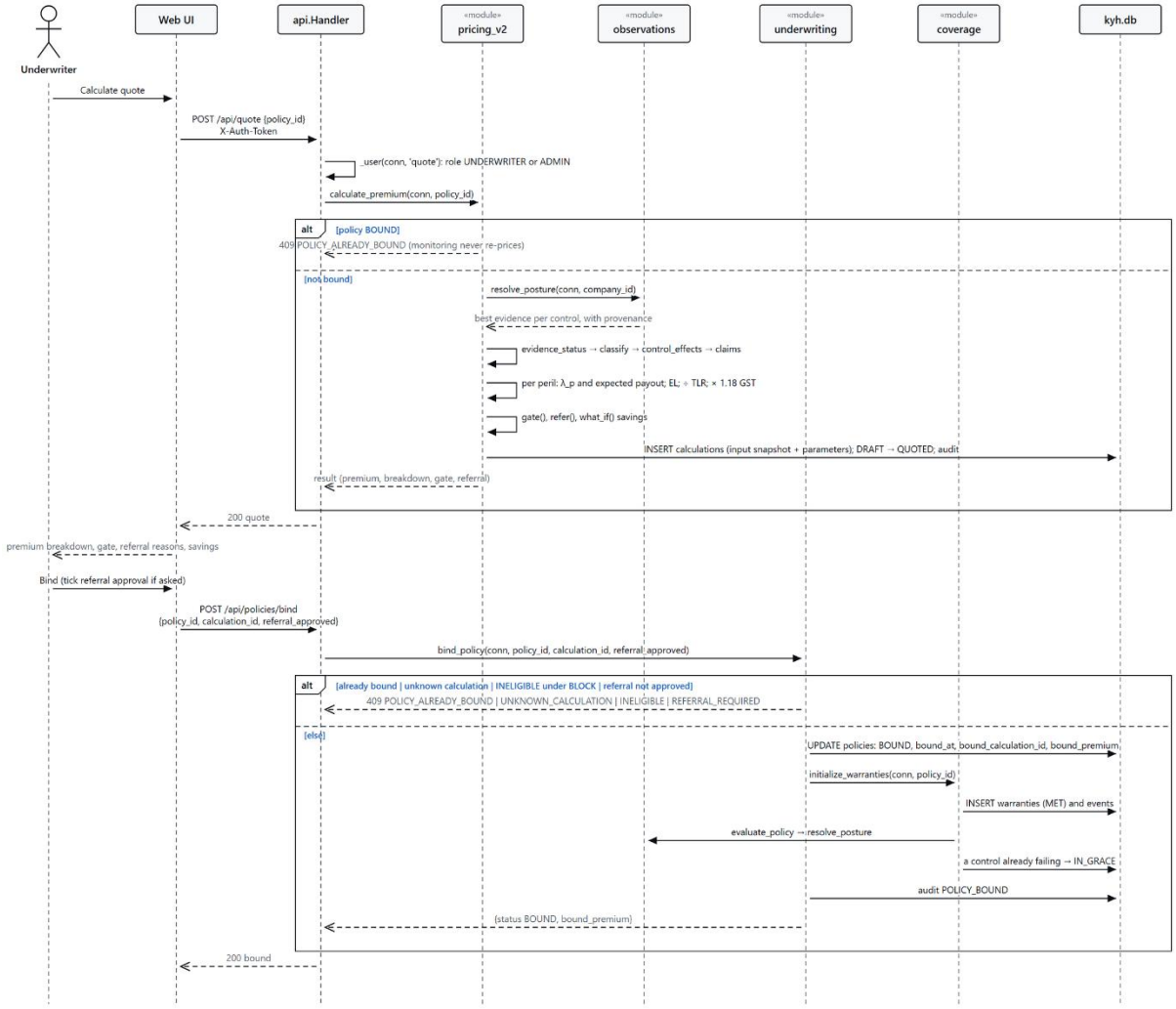


Fig. 10. UML sequence diagram: quote and bind.

6.4 Agent enrolment and a signed report

An agent enrols with a limited-use token. Each report is signed with HMAC-SHA256 over its exact bytes and carries a sequence number, so a tampered or replayed report is rejected.

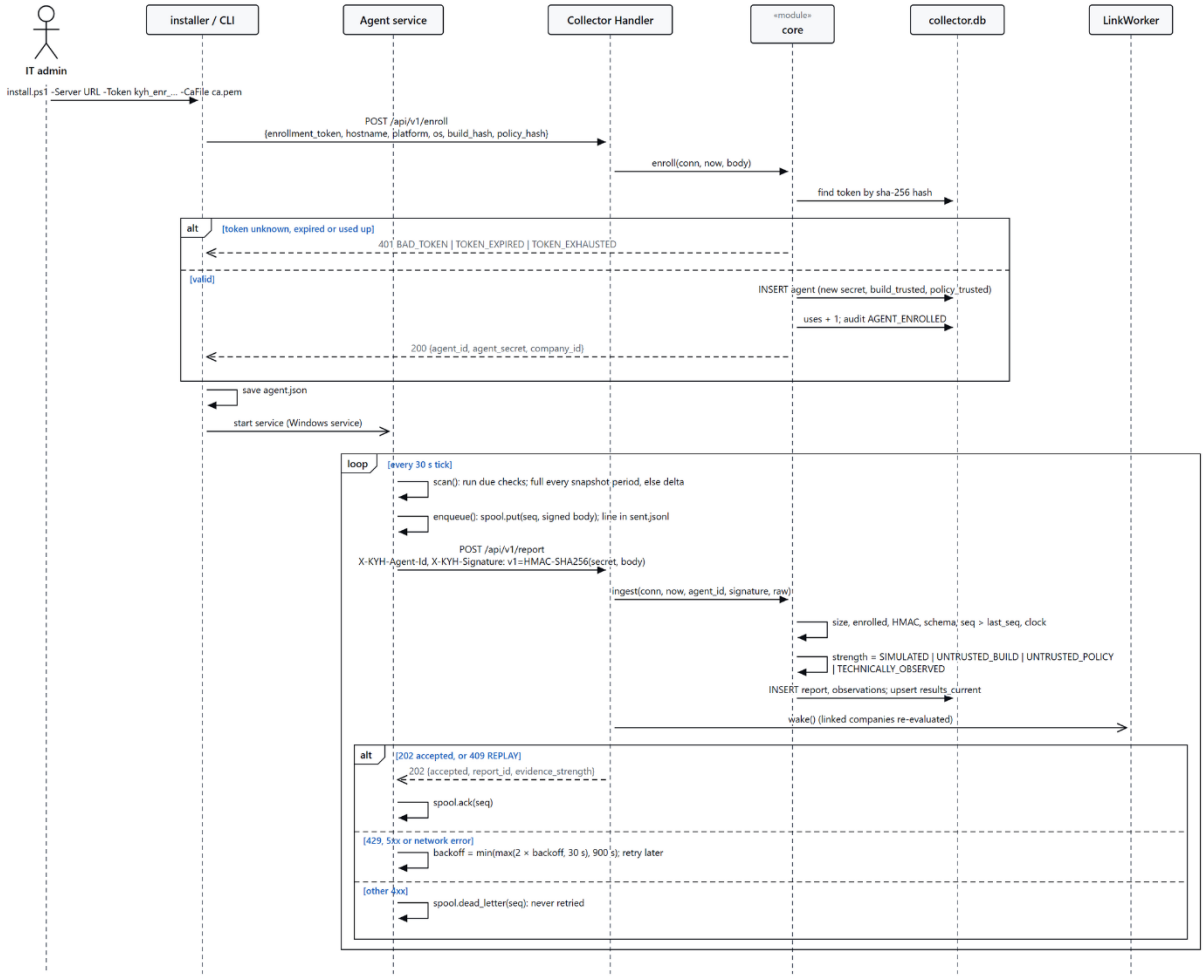


Fig. 11. UML sequence diagram: agent enrolment and a signed report.

7. Algorithms

7.1 Premium calculation

Fig. 12 is the flowchart of a quote. The steps in brief:

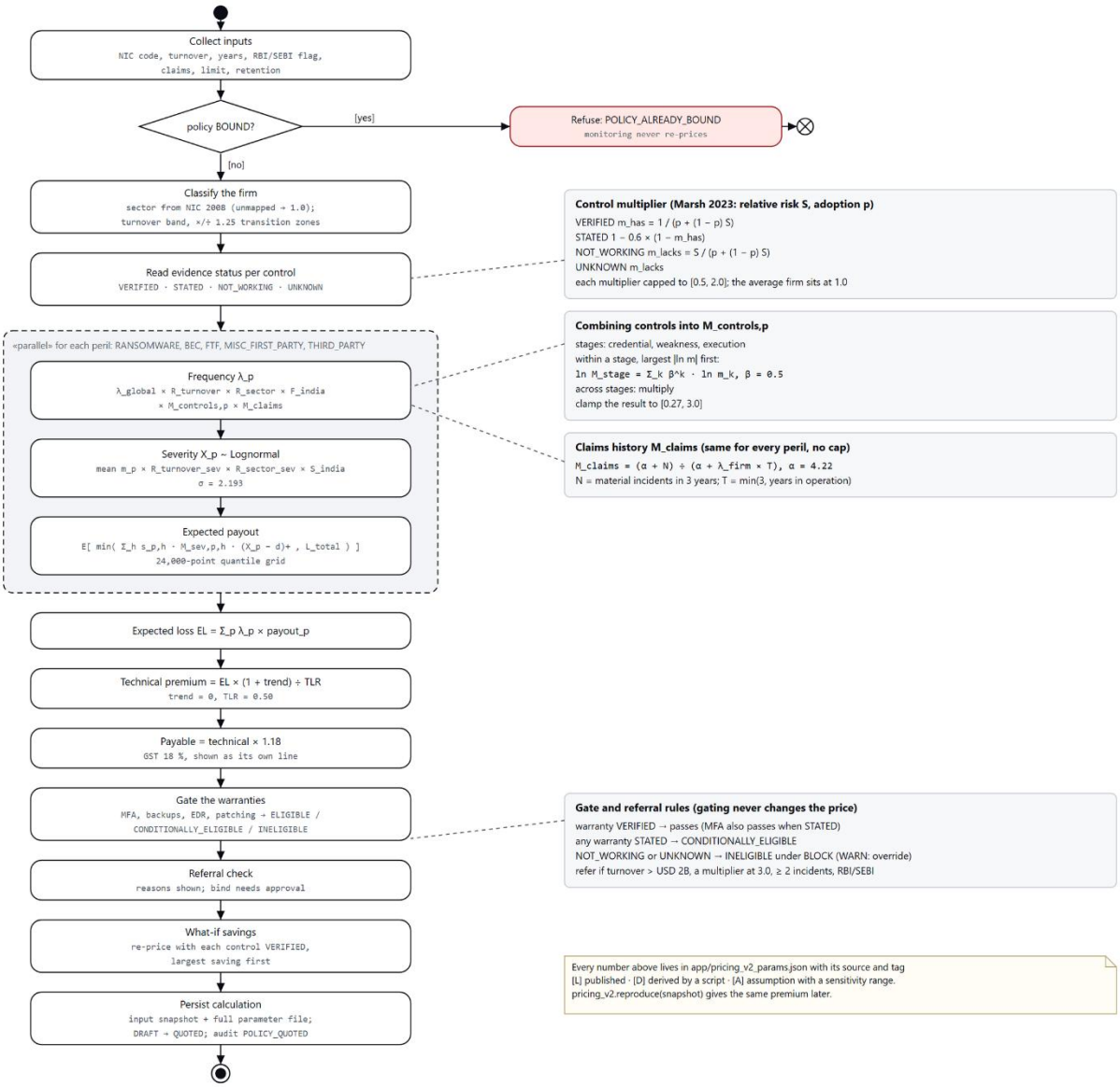


Fig. 12. Flowchart of a quote (UML activity diagram).

Five perils are priced separately: ransomware, business email compromise, funds transfer fraud, miscellaneous first-party loss and third-party liability [3], [4]. For each peril p:

- Frequency: $\lambda_p = \lambda_{global} \times R_{turnover} \times R_{sector} \times F_{india} \times M_{controls,p} \times M_{claims}$.
- Severity: X_p is lognormal with mean $m_p \times R_{turnover,sev} \times R_{sector,sev} \times S_{india}$ and $\sigma = 2.193$.
- Payout: the expected payout $E[\min(\Sigma_h s_{p,h} \times M_{sev,p,h} \times (X_p - d)+, L_{total})]$ is integrated on a 24,000-point quantile grid.

The results combine as follows:

- Expected loss: $EL = \Sigma_p \lambda_p \times payout_p$.
- Technical premium: $EL \times (1 + trend) / TLR$, with trend = 0 and TLR = 0.50.
- Payable premium: technical premium × 1.18, which adds 18% GST.

Control multipliers come from each control's relative risk S and adoption p [5]. A VERIFIED control uses $m_{has} = 1 / (p + (1 - p)S)$, a STATED control earns 60% of that credit, and a NOT_WORKING or UNKNOWN control uses $m_{lacks} = S / (p + (1 - p)S)$. Each is capped to [0.5, 2.0], and the combined multiplier is limited to [0.27, 3.0].

Claims history uses a credibility multiplier [6]: $M_{claims} = (\alpha + N) / (\alpha + \lambda_{firm} \times T)$, with $\alpha = 4.22$, N the material incidents in the last 3 years and T = min(3, years in operation).

Worked example: an IT services firm with ₹150 crore turnover, a ₹5 crore limit and a ₹5 lakh retention gives an expected loss of ₹49,947, a technical premium of ₹99,895 and ₹1,17,876 payable with GST.

7.2 Agent service loop

Fig. 13 shows one tick of the agent service. It runs the checks that are due, sends a delta only when a result changes, sends a heartbeat when one is due, and retries with back-off on a network error.

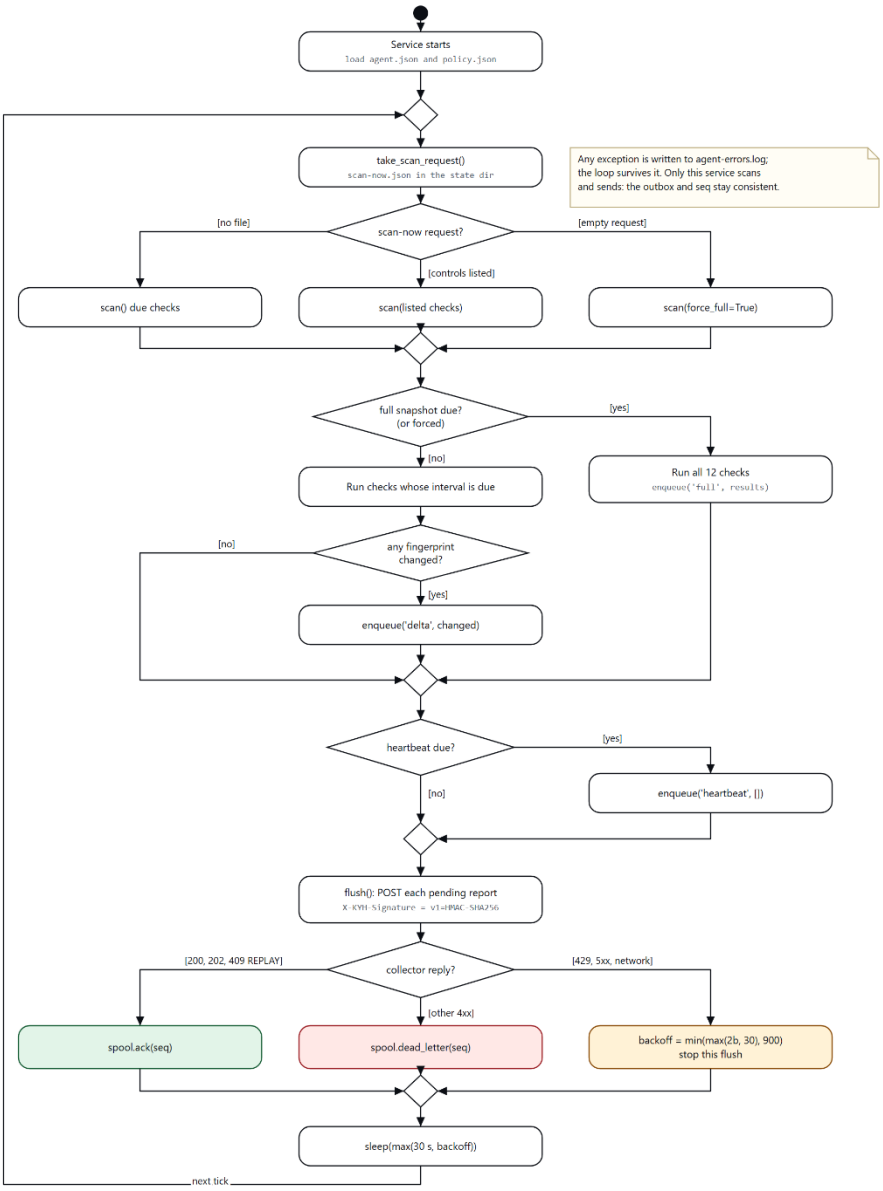


Fig. 13. UML activity diagram of the agent service loop.

7.3 Fleet roll-up

Each host is ONLINE, UNVERIFIED after six hours without a report, or SILENT after 72 hours. A silent host counts as a failure for every control it used to report, so uninstalling the agent cannot hide a failing machine. A host fails a control if any of its checks for that control fails. A company passes a control if at least 95% of the hosts able to observe it pass. If no host can observe a control, as with MFA, nothing is sent for it.

7.4 Evidence resolution

Each input is stored as an observation with one of four evidence strengths: DECLARED, DOCUMENTED, TESTED or TECHNICALLY_OBSERVED. For each control the strongest, most recent evidence wins, and disagreeing sources are flagged.

8. References

[1] Object Management Group, Unified Modeling Language (UML), Version 2.5.1, OMG Document formal/17-12-05, Dec. 2017.

[2] IEEE Standard for Information Technology – Systems Design – Software Design Descriptions, IEEE Std 1016-2009, 2009.

[3] Coalition, Inc., 2025 Cyber Claims Report. San Francisco, CA, USA: Coalition, 2025.

[4] NetDiligence, Cyber Claims Study 2025. Gladwyne, PA, USA: NetDiligence, 2025.

[5] Marsh McLennan, Using Data to Prioritize Cybersecurity Investments. New York, NY, USA: Marsh McLennan, 2023.

[6] H. Bühlmann, “Experience rating and credibility,” ASTIN Bulletin, vol. 4, no. 3, pp. 199–207, 1967.